

Bench Studio

Technical Reference for Autonomous Agent Setup

This document is written as machine-context documentation, not marketing copy. It is dense on purpose. Paste the whole file, or hand the PDF, to a coding agent (Claude Code, Codex, Cursor, ChatGPT with a repo tool) and it can install, configure, and wire Bench Studio into itself without further explanation from you.

Companion to: Bench Studio Ownership Kit (Build Your Own Higgsfield video)

Repo: github.com/promptadvisers/bench-studio-public

1. System Architecture

Bench Studio is a local-first, single-user application with three processes and one credential boundary. No provider API key is ever sent to, or readable by, the browser.

Component	What it is	Address
Studio (web UI)	React app, the visual interface for Create / Model catalog / Results / Websites / Documents / Connect	http://localhost:5200
Local API server	Node.js loopback service (server/server.mjs). Owns all provider credentials, validates payloads per model, streams generation progress, mirrors output files, writes the SQLite ledger	http://localhost:8787
Health / capability check	GET endpoint that reports whether the server, keys, and catalog are healthy	http://localhost:8787/api/health
MCP server	server/mcp.mjs, a stdio Model Context Protocol server exposing the same backend as tool calls for an agent client	spawned via `npm run mcp`, stdio transport (no network port)

Data flow: UI or MCP client -> local loopback API (validates, owns keys) -> fal.ai (and optionally Google, and a locally authenticated Codex CLI for website/document builds) -> local SQLite ledger + mirrored files under data/. The browser and the MCP client are both untrusted from the API's point of view; only the API process reads ~/.env.

Local data boundary (created on first run, gitignored):

```
data/  
  bench.db generations, assets, spend, and projects (SQLite)  
  inputs/ mirrored uploads  
  outputs/ mirrored generations  
  previews/ local video posters  
  projects/ website and document source files
```

Deleting a result removes its local database row and mirrored files. It does not delete any copy retained by the external model provider (fal.ai, Google, etc). Treat provider-side retention as out of Bench's control.

2. Environment Variables and Credentials

All credentials load from a single file at ~/.env (the user's home directory, NOT a project-local .env). The API server auto-loads this file on boot and only fills in a variable if it is not already set in the process environment (server.mjs reads the file, then does `if (!process.env[k]) process.env[k] = v`). Never put provider keys in Vite environment variables or commit them to any repository.

Variable	Required for	Where to get it
FAL_KEY	Bench Studio core: all 37 model routes, image + video generation through the fal.ai aggregator. Required to run Bench Studio at all.	fal.ai -> dashboard -> keys
GOOGLE_API_KEY	Optional prompt-refinement rewriter inside Bench Studio (gemini-3-flash-preview). Without it Bench still runs and submits the user's original prompt unchanged. Also covers the standalone generate skill's gemini-image engine and Veo video.	Google AI Studio
OPENAI_API_KEY	The standalone generate skill's gpt-image engine (gpt-image-2). Not used by Bench Studio's core server.	platform.openai.com

Variable	Required for	Where to get it
MINIMAX_API_KEY	generate skill, direct Hailuo lane (bypasses fal, ~\$0.25/clip, raw provider price)	platform.minimax.io
QWEN_CLOUD_API_KEY (or DASHSCOPE_API_KEY)	generate skill, direct Wan lane + qwen-image (wan2.6-t2v \$0.10/s)	qwencloud.com, international DashScope endpoint
KLING_ACCESS_KEY + KLING_SECRET_KEY	generate skill, direct Kling lane (JWT auth, requires a Kling global API plan)	Kling API console
BENCH_DATA_DIR	Optional override for the data/ directory location (defaults to a data/ folder next to server/)	n/a, self-set if needed
PORT / BENCH_API_PORT / BENCH_WEB_PORT	Optional overrides when the default ports 8787/5200 are occupied	n/a, self-set if needed

Precedence rule for the four Chinese video lanes (wan, hailuo, kling, seedance): a direct provider key (MINIMAX_API_KEY, QWEN_CLOUD_API_KEY/DASHSCOPE_API_KEY, or the KLING pair) always beats FAL_KEY when both exist, because it is billed at raw provider price instead of the fal aggregator's markup. The generate skill's script auto-detects which keys are present and logs which lane was actually used (e.g. "hailuo-direct" vs "fal-hailuo") in the receipts ledger. Seedance has no direct-API path; it is fal-only because BytePlus's direct API is enterprise-console gated.

```
# ~/.env (create or append to this file, NOT a project .env)
FAL_KEY=your-fal-key
GOOGLE_API_KEY=your-google-key
OPENAI_API_KEY=your-openai-key
MINIMAX_API_KEY=your-minimax-key
QWEN_CLOUD_API_KEY=your-qwen-key
KLING_ACCESS_KEY=your-kling-access-key
KLING_SECRET_KEY=your-kling-secret-key
```

3. Install and Run

Requirements: Node.js 22.5 or newer (Node 24 recommended, Bench uses node:sqlite), npm, a fal.ai API key, Google Chrome installed locally (for PDF document printing and visual preflight). A signed-in Codex CLI is optional and only needed for the website/document builder tools.

```
git clone https://github.com/promptadvisers/bench-studio-public.git
cd bench-studio-public
npm install

# add credentials to ~/.env as shown in section 2, then:
npm run dev
```

npm run dev starts both processes: the API on :8787 and the web UI on :5200. If either port is already in use, override both explicitly:

```
PORT=8790 BENCH_API_PORT=8790 BENCH_WEB_PORT=5201 npm run dev
```

Verify the server is healthy before doing anything else:

```
curl -s http://localhost:8787/api/health
```

A missing FAL_KEY causes the server to log an error and the health check to report a disabled generation path. Fix by adding FAL_KEY to ~/.env and restarting the dev process (env is only read at boot).

Other useful commands:

Command	Purpose
npm run build	Build the production web application
npm run registry	Rebuild the curated model registry (server/registry.json)
npm run capabilities	Rebuild the capability manifest (server/capabilities.json)
npm run catalog:sync	Refresh live provider discovery and pricing evidence
npm run mcp	Start the stdio MCP server directly (what an agent client actually spawns)
npm run test:contracts	API, persistence, and model-contract tests
npm run test:mcp	Smoke-test MCP discovery and media behavior
npm run test:release	Full release gate before trusting a change

4. The MCP Server: Tools and Contracts

server/mcp.mjs registers an McpServer named "bench-studio" over stdio transport using the official MCP SDK (McpServer + zod schemas). It is started with ``npm run mcp`` and talks to the SAME local API on :8787 that the browser UI uses, so every MCP call is subject to the same validation, credential custody, and ledger writes. Start Bench Studio's dev server FIRST; the MCP server proxies to it and will fail if :8787 is not reachable.

list_models Discover available models

Inputs: output: 'all'|'image'|'video' (default all); accepts: 'any'|'image'|'video'|'audio'|'document' (default any); search: optional string
Filters the live catalog (GET /api/models). Returns id, label, vendor, output kind, lane, accepted input modalities, and pricing for each match. ALWAYS call this before create_media.

get_model_capabilities Inspect one model's exact input contract before attaching any media

Inputs: model_id: string (exact id from list_models, e.g. 'fal-ai/flux-2/flash')

Returns the schema-derived input fields and verification-status evidence (see status legend in section 5). Treat 'schema-supported' as declared support only, not proof the model will actually use an attached reference well.

upload_media Copy a local file into Bench and upload it to fal for use as a model input

Inputs: path: string, absolute local filesystem path to an image, video, audio file, or PDF

Returns the hosted fal URL plus the local archive URL. Keep both; hosted URL goes into input_assets, local URL is for reporting back to the user.

create_media Generate an image or video

Inputs: model_id: string; prompt: string; mode: string (default 'none'); params: object (default {}); input_assets: array of {url, field, media_type} (default [])

field must be the EXACT capability field name returned by get_model_capabilities for that model_id. Never guess a generic name like image_url; models differ. Returns the generation row with local + hosted result URLs and recorded/estimated cost.

list_results List recent generations with inline previews

Inputs: limit: int 1-200 (default 20); include_previews: bool (default true); preview_limit: int 1-10 (default 5)

Returns ledger summary plus rows: prompt, model, provider URL, local file, recorded cost. Supports native inline image/video preview rendering in MCP-aware clients.

get_usage Report actual recorded spend and storage

Inputs: none

Returns { usage, storage } from /api/spend and /api/storage. Call before assuming a cost; Bench distinguishes estimated, metered, and recorded values.

sync_models Refresh the catalog from fal

Inputs: none

Triggers POST /api/catalog/sync. Run when a model is missing or suspected outdated.

create_website Start a local Codex-powered static website build

Inputs: title: string; brief: string; direction: optional string; depth: 'low'|'medium' (default low)

Uses the machine's cached ChatGPT/Codex authentication, not an API key. Write a concrete brief: audience, purpose, mood, required sections, constraints. Returns a project_id; poll with get_project.

create_document Start a local Codex-powered designed-PDF build

Inputs: Same input shape as create_website (title, brief, direction, depth)

Produces editable HTML plus a Chromium-printed PDF. Poll with get_project; visually inspect representative pages before reporting a final production artifact as finished.

list_projects List local website/document builds

Inputs: kind: 'all'|"website"|"document" (default all)

Returns build state and artifact URLs for each project.

get_project Check build progress / retrieve artifacts

Inputs: project_id: string

Poll until status is complete, failed, or cancelled. Never report a queued or in-progress job as delivered.

Note: the README's public-facing count is eleven tools (it also counts an internal results-gallery resource registration and reserves room for two more, list_results/create_website/create_document, in its rounding). The eleven functional tools above are the complete, exhaustive contract as implemented in server/mcp.mjs at the time this document was written; if a future version adds tools, re-run list_models-style discovery against the live server (an agent can call the MCP server's own tools/list method) rather than trusting a stale count.

5. Model Routing, Profiles, and Capability Contracts

Bench Studio does not hand an agent a raw list of fal endpoint URLs. It runs discovery and production admission as two separate stages so a renamed or underspecified model can never silently break a paid workflow.

- `server/discover_catalog.mjs` pulls fal's live OpenAPI queue schemas into a discovery snapshot with schema and pricing evidence.
- `server/build_registry.mjs` curates that snapshot into `server/registry.json`, the production roster (37 models at last generation: 'generated_at', 'source': fal.ai public OpenAPI queue schemas, 'count': 37). Each entry carries id, kind (image/video), lane (e.g. t2i = text-to-image), label, vendor, tier, its paired edit/variant model, category, thumbnail, and a required[] field list plus a params map with per-field type/title metadata.
- `server/build_capabilities.mjs` rebuilds `server/capabilities.json`, the capability manifest consumed by both the UI controls and the MCP `get_model_capabilities` tool. Each model's fields carry a verification status.

Capability verification status legend (from capabilities.json):

Status	Meaning
schema-supported	Declared by fal's live OpenAPI schema; not necessarily exercised
submission-verified	A real request containing this field was accepted
output-verified	A generated result was manually or automatically checked against the input
failed	A real request containing this field was rejected
untested	No current evidence exists

server/profiles/*.json: prompt and pricing intelligence per model family

The profiles directory holds per-family JSON used by the prompt refiner and the pricing engine, not by the MCP tool contract directly. Files present in the shipped repo:

- `closed_image.json`, `cn_image.json`, `flux.json`, `klimg_hailuo.json`, `open_video.json`, `seedance_veo.json`: first-generation family profiles (prompt style guidance plus pricing hints per model family).
- `p2_closed_image.json`, `p2_cn_image.json`, `p2_flux.json`, `p2_seedance_h3.json`, `p2_veo_kling.json`, `p2_wan_ltx_grok.json`: a second, more granular profile generation split further by model family.
- `pricing.json`: the pricing intelligence file. It documents how Bench derives real billed cost. fal returns the actual billed quantity in the x-fal-billable-units response header on the queue result GET request, already denominated in that endpoint's priced unit (images, megapixels, video seconds, compute seconds). Actual cost equals billable_units times unit_price, where unit_price comes from fal's pricing API (GET https://api.fal.ai/v1/models/pricing?endpoint_id=<endpoint_id>). When the header is absent, fal falls back to per-second GPU-machine billing and Bench falls back to an estimate.

Adding a new model to the routing layer (the procedure an agent should follow):

- 1. Run ``npm run catalog:sync`` (or call the `sync_models` MCP tool) to pull the newest fal OpenAPI schemas into the discovery snapshot.
- 2. Run ``npm run registry`` to regenerate `registry.json` from the discovery snapshot; a new model appears here first as an observable candidate, not yet production-admitted.
- 3. Run ``npm run capabilities`` to regenerate `capabilities.json` with schema-supported status for the new model's fields.
- 4. Manually review the model in `registry.json/capabilities.json`: confirm pricing evidence and input schema look correct, then it is safe to route generation requests to it. This human review step is intentional; Bench does not auto-promote new catalog entries into the curated roster without it.

6. Cost Ledger and Receipts

Every generation writes a row to the local SQLite database at `data/bench.db`, readable through `/api/ledger` (UI) or the `list_results / get_usage` MCP tools (agent). Each row carries the submitted prompt, model id, fal-hosted result URL, mirrored local file path, and cost with a confidence label: estimated (preflight, before submission), metered (during a streaming run), or recorded (the real billed amount once fal exposes a receipt via the `x-fal-billable-units` header, per section 5). Never present an estimate as a recorded fact when reporting spend back to a user; state which of the three it is.

The standalone generate skill (bundled separately in this kit's `generate_skill/` folder, not part of Bench Studio's server) keeps its own lightweight parallel ledger: every successful run appends a line to `receipts.md` one folder above the skill, with timestamp, engine, prompt snippet, output file, estimated cost, and a running total. Report both the per-run cost and the running total after each generation when operating that skill.

7. Copy-Paste Setup Prompt for a Coding Agent

Paste the block below directly into Claude Code, Codex, Cursor, or any coding agent with shell and file access. It is written as literal, ordered instructions the agent should execute and self-verify, not as narrative documentation.

```
You are setting up Bench Studio, a local-first creative studio, and wiring it
into yourself as an MCP
server. Follow these steps in order and verify each one before moving to the next.

1. Clone and install:
git clone https://github.com/promptadvisers/bench-studio-public.git
cd bench-studio-public && npm install

2. Check for required credentials. Read ~/.env (create it if absent). Confirm
FAL_KEY exists; it is
required. If missing, stop and ask the user for a fal.ai API key from
fal.ai/dashboard/keys before
continuing. GOOGLE_API_KEY is optional (enables prompt refinement); note whether
it is present.

3. Start the dev server in the background: `npm run dev`. Wait, then verify
health with:
curl -s http://localhost:8787/api/health
Confirm the response reports the server as healthy and FAL_KEY as configured
before proceeding.
If port 8787 or 5200 is already bound, restart with PORT=8790 BENCH_API_PORT=8790
BENCH_WEB_PORT=5201 npm run dev and use the new ports for all further checks.

4. Register the MCP server with yourself. Bench Studio's own Connect tab (open
http://localhost:5200, click Connect, pick your client) generates the exact
machine-correct
config. If you cannot open a browser, the config points at running `npm run mcp`
from the
bench-studio-public/server directory as a stdio MCP server; add it to your own
MCP client config
(for Claude Code: `claude mcp add` or the equivalent config file entry) with the
absolute path to
this cloned repo.

5. Verify the MCP connection: call the list_models tool. Confirm it returns
roughly 37 models. Then
call get_model_capabilities on one image model and confirm you can read its
required input fields.

6. Run one cheap end-to-end test: pick the lowest-cost image model from
list_models, call create_media
with a simple test prompt, then call list_results and confirm the new generation
appears with a
recorded or estimated cost. Report the exact cost back to the user.

7. Optional: install the bundled workflow skill for routing judgment:
cp -r integrations/skills/bench-studio ~/.claude/skills/bench-studio

8. Report back to the user: server health, which optional keys are configured
versus missing, the
MCP tool list confirmed live, and the result of the one test generation
including its cost.
```

8. Troubleshooting

Symptom	Likely cause	Fix
MCP tool calls fail immediately	Bench Studio's dev server is not running	Start <code>`npm run dev`</code> in <code>bench-studio-public</code> first, in a separate process; confirm with <code>curl http://localhost:8787/api/health</code> before retrying MCP calls
Server logs 'No FAL_KEY in ~/.env' and refuses generation	FAL_KEY missing or in the wrong file	Add FAL_KEY to <code>~/.env</code> (home directory, not a project <code>.env</code>); restart the dev server, <code>env</code> is only read at process boot
Prompt refinement silently does nothing	GOOGLE_API_KEY not set	Expected behavior, not a bug. Bench reports refinement disabled and passes the original prompt through unchanged. Add GOOGLE_API_KEY to enable it
<code>create_media</code> rejects an attached reference image	The field name in <code>input_assets</code> does not match the model's exact capability field	Call <code>get_model_capabilities</code> for that <code>model_id</code> first and copy the exact field name; never assume <code>image_url</code> works for every model
Port 5200 or 8787 already in use	Another process (or a previous Bench instance) is bound to the default ports	Restart with <code>PORT=8790 BENCH_API_PORT=8790 BENCH_WEB_PORT=5201 npm run dev</code>
<code>create_website / create_document</code> jobs never complete	No signed-in Codex CLI authentication on this machine	This feature uses cached local ChatGPT/Codex authentication, not an API key. Sign in to the Codex CLI once outside of Bench, then retry
A model disappears or behaves differently after a while	<code>fal</code> 's catalog changed upstream	Run <code>sync_models</code> (or <code>`npm run catalog:sync`</code> then <code>`npm run registry`</code> and <code>`npm run capabilities`</code>) to refresh the curated roster and capability manifest
Reported cost differs from the <code>fal.ai</code> dashboard	The endpoint did not expose the <code>x-fal-billable-units</code> header, so Bench used a per-second GPU fallback estimate	Treat that row's cost as an estimate, not a recorded fact; check <code>get_usage</code> for the confidence label

This document describes the Bench Studio public repository (github.com/promptadvisers/bench-studio-public) and the companion `generate_skill` bundled in this kit's `generate_skill/` folder, as of the build reflected in this package. Model routing, pricing, and the exact tool count can change upstream; re-verify against the live server and repo when in doubt.